



# Python Control Flow

By PrudviKrishna



# If – elif - else

## How they work together

When you use these statements together, Python evaluates them in order:

## Syntax :

if **condition1**:

# Block of code executed if condition1 is True

elif **condition2**:

# Block of code executed if condition1 is False and condition2 is True

else:

# Block of code executed if both condition1 and condition2 are False

## Example code:

```
x = 10
```

```
if x > 10:
```

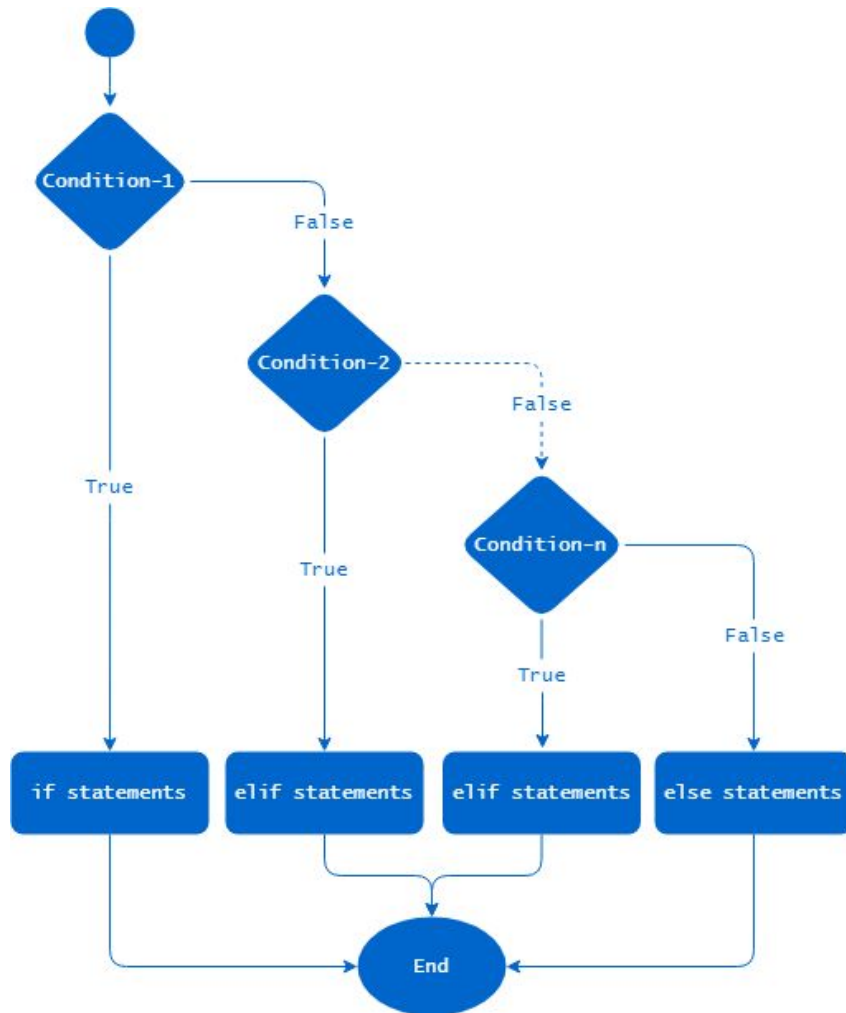
```
    print("x is greater than 10")
```

```
elif x == 10:
```

```
    print("x is equal to 10")
```

```
else:
```

```
    print("x is less than 10")
```



Why to use if-elif-else, cannot it be achieved with multiple if conditions

# Loops – While

## While Loop

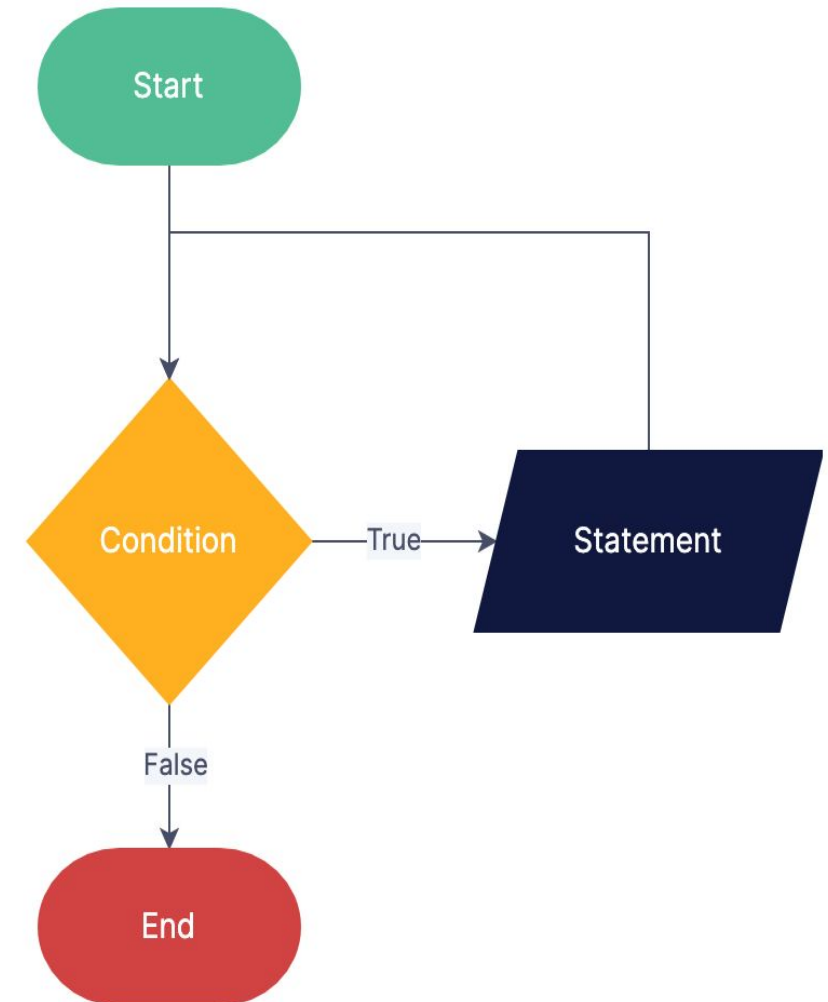
A while loop repeats a block of code as long as a specified condition is True. Once the condition becomes False, the loop stops.

### Syntax:

```
while condition:
    # Block of code to execute
```

### Example of a while loop:

```
i = 0
while i < 5:
    print(i)
    i += 1 # Increment i by 1 in each iteration
```



# Loops - For

## for Loop

A for loop in Python is used for iterating over a sequence (like a list, tuple, dictionary, set, or string) or a range of numbers.

It allows you to execute a block of code repeatedly, once for each item in the sequence.

### Syntax:

```
for variable in iterable_collection:
    # Block of code to execute
```

### Example Code:

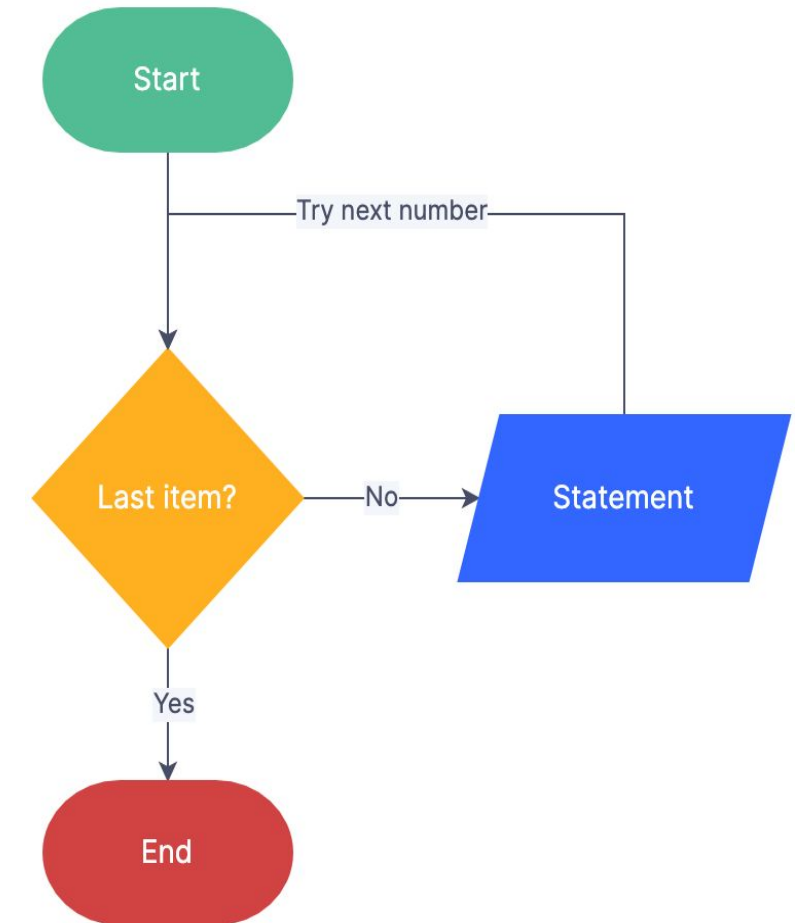
```
fruits = ["apple", "banana", "cherry"]
```

### # Example 1 Iterating over a list

```
for fruit in fruits:
    print(fruit)
```

### #Example2 using enumerate

```
for index, fruit in enumerate(fruits):
    print(f"Index: {index}, Fruit: {fruit}")
```



# Loop Control Statements - Break

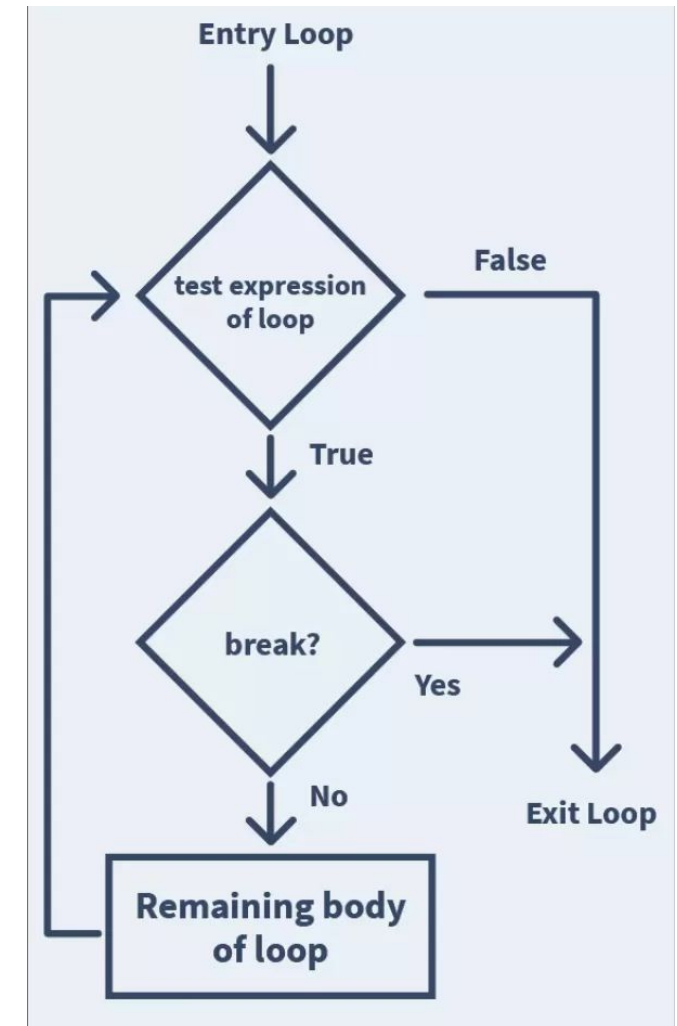
## break:

The break statement is used to exit the loop immediately, regardless of the loop's condition.

Once break is encountered, the control jumps out of the loop, and the program continues with the next statement after the loop.

## Example code:

```
for i in range(5):
    if i == 3:
        break # Exits the loop when i is 3
    print(i)
```



# Loop Control Statements - Continue

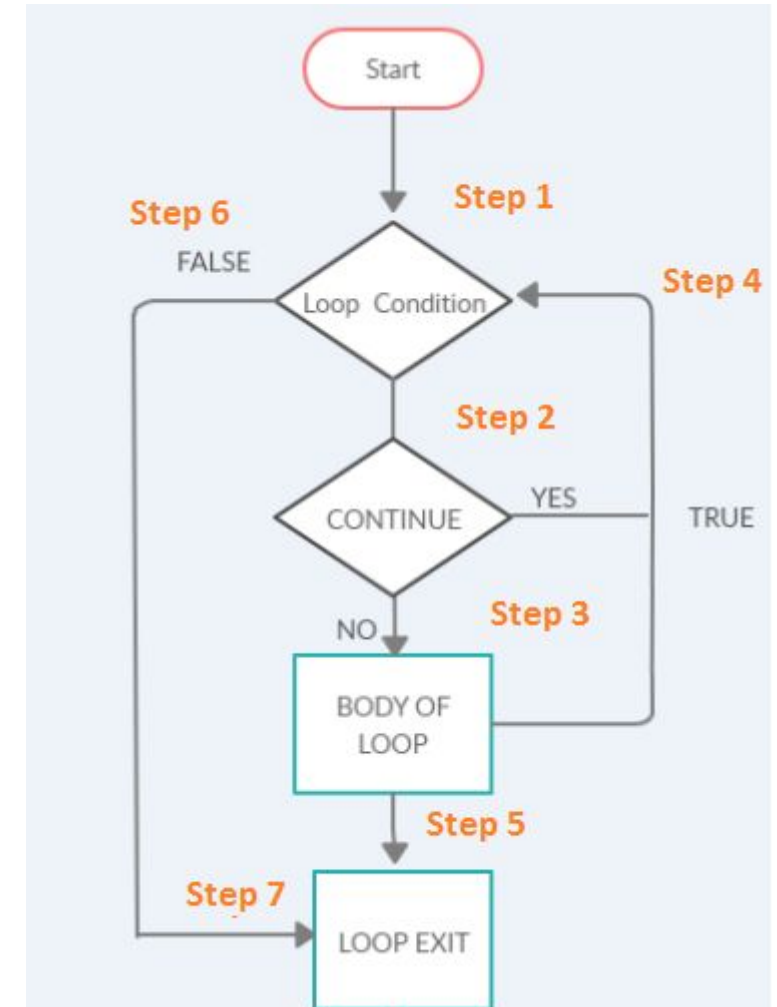
## continue:

The continue statement is used to skip the current iteration of the loop and move to the next iteration.

The loop continues running, but anything after continue within that iteration is ignored.

## Example code:

```
for i in range(5):
    if i == 3:
        continue # Skips the iteration when i is 3
    print(i)
```



# Loop Control Statements - Pass

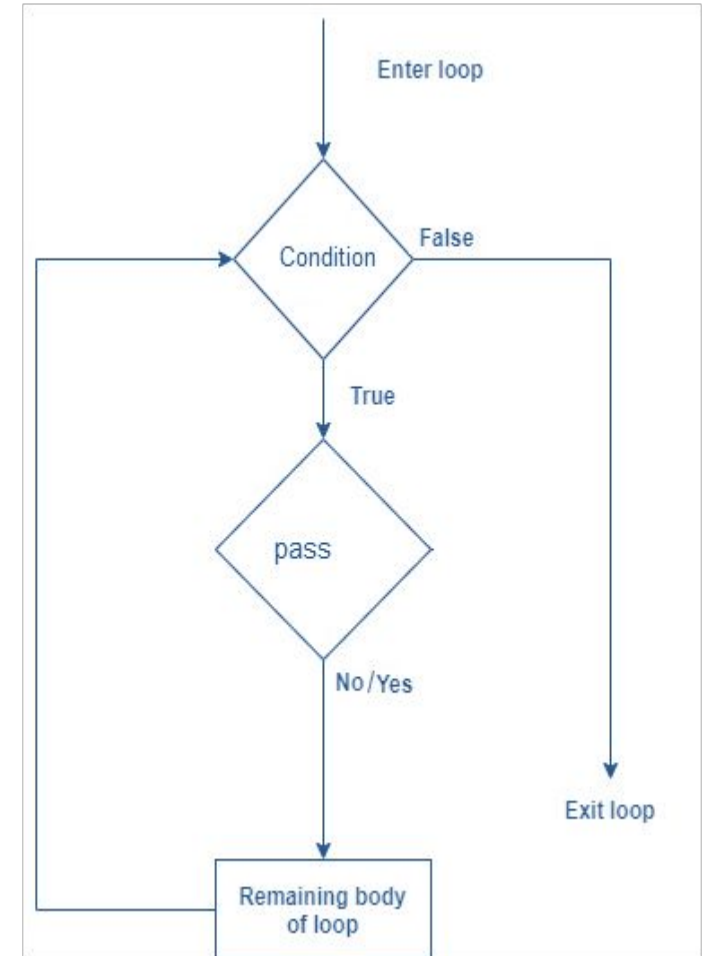
## **pass:**

The pass statement is a placeholder in loops, functions, or conditional statements.

It doesn't do anything and is often used as a placeholder for code that will be written later.

## **Example code:**

```
for i in range(5):  
    if i == 3:  
        pass # Does nothing, just a placeholder  
    print(i)
```



A large, stylized circular graphic composed of multiple concentric, slightly offset rings in shades of blue, green, and purple, creating a sense of depth and movement. The text 'Thank you' is centered within this graphic.

**Thank you**